

Building AI APIs and Backends with FastAPI

Topics

- e! REST API fundamentals: HTTP methods, status codes, request/response lifecycle
- e! FastAPI project structure: routers, models, and dependency injection
- e! Defining `pydantic` models for request validation and response schemas
- e! Building `async` FastAPI endpoints for LLM-powered routes
- e! Streaming LLM responses using Server-Sent Events (SSE)
- e! Handling file uploads in FastAPI for document processing pipelines
- e! Running and testing FastAPI with `uvicorn` and Swagger UI

Learning Objectives

By the end of this lesson, you will be able to:

- e! Explain REST APIs and the HTTP request–response lifecycle.
- e! Build FastAPI applications using routers, models, and dependency injection.
- e! Validate requests and responses using `pydantic` models.
- e! Develop asynchronous and streaming AI APIs with FastAPI.
- e! Run, test, and document APIs using `uvicorn` and Swagger UI.

REST API Fundamentals: HTTP Methods, Status Codes, Request/Response Lifecycle

Why AI Applications Need Frameworks?



AI systems developed as Python scripts cannot easily serve users.



Challenges



No browser access



No remote requests



No application integration



No production deployment

AI Application Workflow

AI applications usually follow a structured flow from user input to backend processing and final response delivery.



User interacts with a web interface



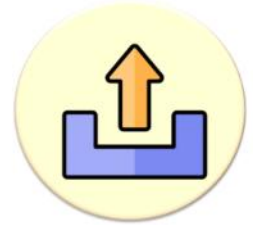
Front-end sends input to an API backend



Backend validates and processes the request

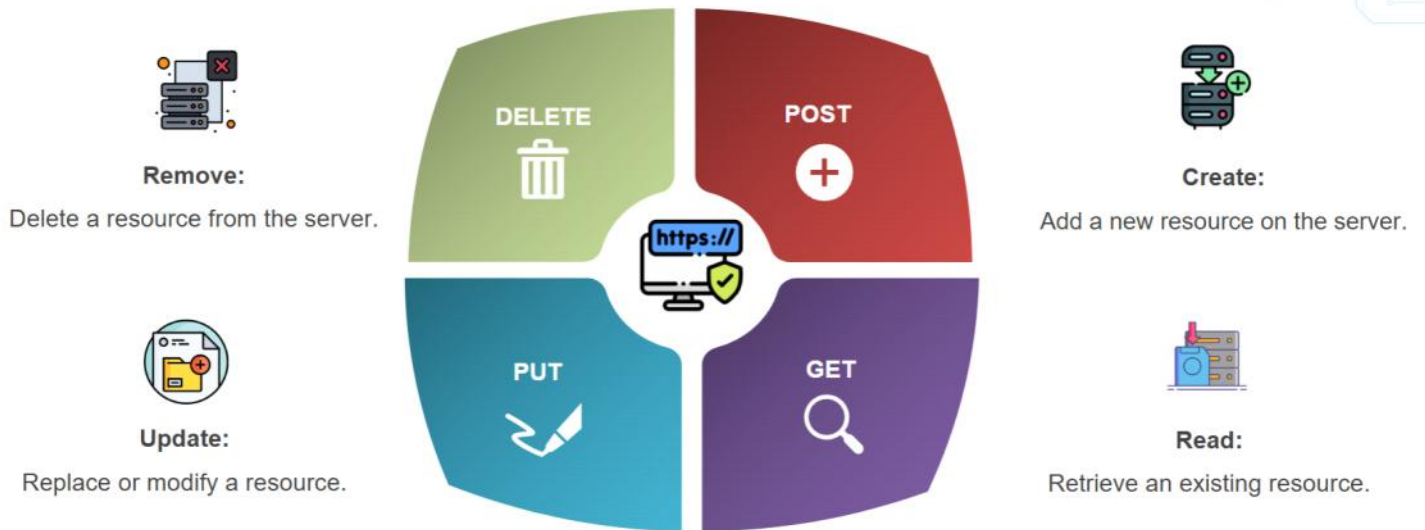


LLM or AI service generates the response

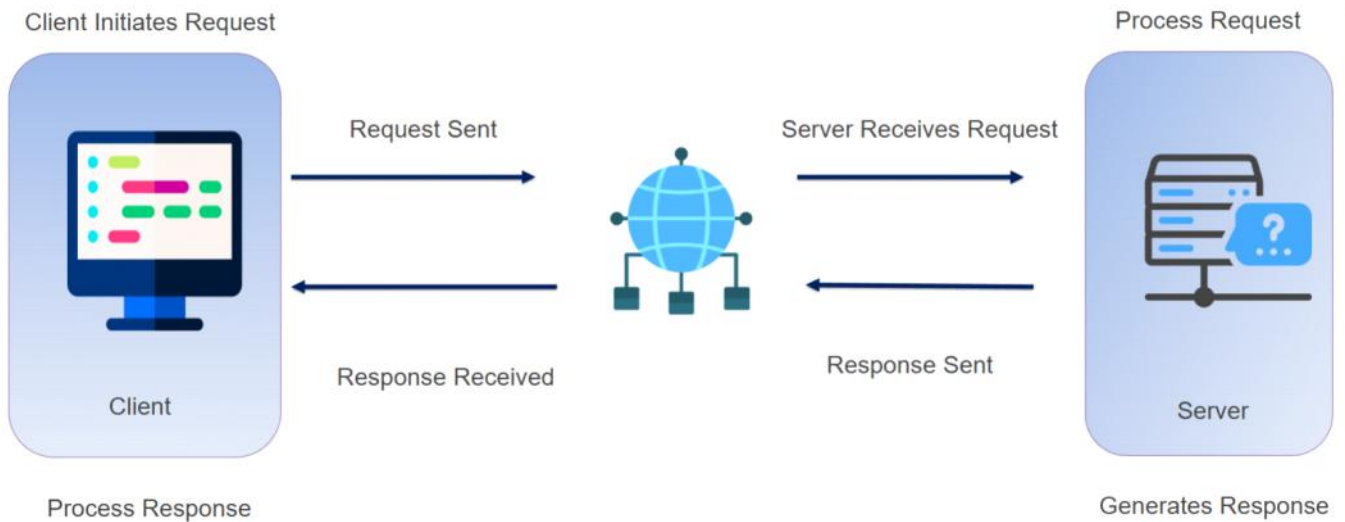


Output is returned and displayed to the user

HTTP Methods



Request-Response Lifecycle



HTTP Status Codes

Code	Meaning	What it tells the client
200	OK	Request completed successfully.
201	Created	New resource created successfully.
400	Bad Request	Invalid request sent by client.
401	Unauthorized	Authentication is required.
404	Not Found	Requested resource does not exist.
500	Server Error	Server encountered an internal error.

FastAPI Project Structure: Routers, Models, and Dependency Injection

Introducing FastAPI

FastAPI is a modern, high-performance Python web framework for building APIs, built on standard type hints.



Modern Python web framework



High-performance API development



Built for asynchronous applications



Widely used in AI systems

 **FastAPI**



Routers and Models

Routers

Organise endpoints by feature. Use **lowercase route paths**: /chat, /documents, /agents.

```
routes/chat.py

from fastapi import APIRouter

router = APIRouter()

@router.post("/chat")
async def chat():
    return {"reply": "..."}

```

Models

Define the data structures your app moves around:
User, Document, ChatSession.

```
models.py

from pydantic import BaseModel

class Document(BaseModel):
    id: int
    title: str
    content: str

```

Dependency Injection

A pattern where a function **receives** the services it needs from an external source, instead of creating them itself.

```
main.py

from fastapi import Depends, FastAPI

app = FastAPI()

def get_db():
    # 1 · Dependency is resolved first
    return Database()

@app.get("/agents")
async def list_agents(db=Depends(get_db)):
    # 2 · db is injected into the endpoint

    return db.all_agents() # 3 · Body runs using the injected db

```



Dependency Function

`get_db()`: Creates and provides the database instance.



Endpoint Function

`list_agents(db)`: Receives the dependency and processes the request.



HTTP Response

Returns the API response to the client.

Defining `pydantic` Models for Request Validation and Response Schemas

Introduction to pydantic

pydantic is a Python data-validation library that uses type hints to define and enforce data schemas.

Type-hint driven: Validation defined with ordinary Python types.

Request & response schemas: Describes exactly what goes in and comes out.

Auto OpenAPI docs: Schemas generate interactive documentation.

Rejects bad input early: Invalid requests fail before your logic runs.

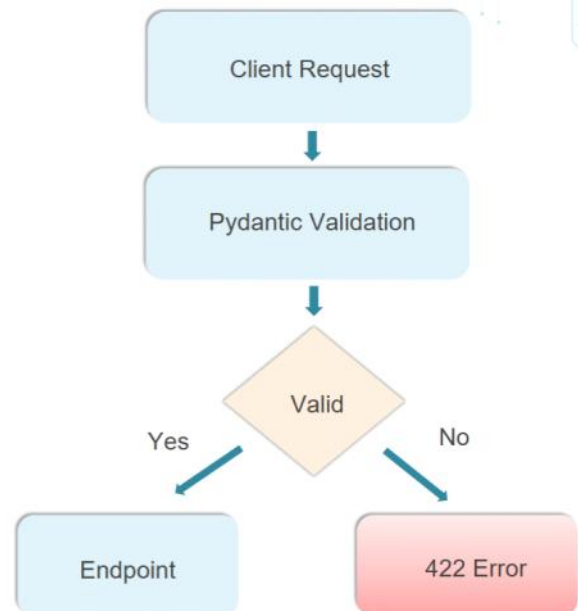


Request Validation

```
schema.py
from pydantic import BaseModel

class ChatRequest(BaseModel):
    prompt: str
```

```
request.json: matching body
{
  "prompt": "Explain AI Agents"
}
```



Response Schemas

RAG Use case

A retrieval augmented endpoint returns the generated **answer** together with the **sources** it drew from so the client can show citations.

```
schema.py

from pydantic import BaseModel
class RAGResponse(BaseModel):
    answer: str
    sources: list[str]
```

```
response.json: matching output

{
  "answer": "An AI agent plans and acts
toward a goal.",
  "sources": [
    "docs/agents.pdf",
    "docs/planning.pdf"
  ]
}
```

Building `async` FastAPI Endpoints for LLM-Powered Routes

Why `async` Matters for AI Applications?

AI backends spend most of their time **waiting**. `async` lets one process handle many concurrent waits instead of blocking on each one.

Where The Time Actually Goes, Waiting On



LLM APIs

Token generation takes seconds per call.



Vector databases

Similarity search over embeddings.



Search services

External retrieval and web lookups.



External tools

Third-party APIs the agent calls.

async Programming in FastAPI

async

Marks a function as a coroutine: it can pause and resume.

await

Pauses the coroutine while a slow call runs, freeing the process to serve other requests, then resumes with the result.

```
async_example.py

async def task():
    result = await fetch()
    return result
```

Best Practice

Use **await** for every network or model call so the event loop is never blocked.

async Endpoints in FastAPI

Works With Any *async* Llm Provider



OpenAI



Anthropic

9

Groq

```
main.py

@app.post("/chat")
async def chat():
    response = await llm.generate()
    return response
```

Streaming LLM Responses Using SSE

Why Stream LLM Responses?

Traditional: Wait For The Full Response

The user stares at a spinner until the entire answer is ready, then it appears all at once.

Streaming: Token By Token

Words appear as they are generated, so the answer unfolds live in front of the user.

Benefits Of Streaming:



Reduced Perceived Latency

The wait feels far shorter.



Incremental Delivery

Content arrives in pieces, not one block.



Continuous Feedback

The user sees progress the whole time.

What is Server-Sent Events (SSE)?

SSE is a **one-way, persistent HTTP connection** over which the server continuously pushes events to the client.



Server

Persistent Connection



Client

Characteristics

- One-way communication
- Persistent connection
- Real-time updates

Use Case

- LLM streaming
- Notifications
- Progress updates

Streaming in FastAPI

```
sse.py

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

async def event_stream():
    async for token in llm.stream():
        yield f"data: {token}\n\n" # SSE event frame

@app.get("/stream")
async def stream():
    return StreamingResponse(
        event_stream(),
        media_type="text/event-stream",
    )
```

Media_type

"text/event-stream" tells the client this is an SSE stream, not a normal body.

Event Framing

Each event is a data: ... line followed by a blank line: the \n\n terminator.

Best Practice

Yield one token per event so the client renders text as it streams in.

SSE in Agentic AI



User Prompt



LLM Stream



FastAPI



UI Updates

Applications

Conversational AI

Chat replies that stream as they generate.

Document Summarization

Summaries that build up progressively.

AI Copilots

In-app assistants with live output.

Research Assistants

Findings streamed as they are gathered.

Handling File Uploads in FastAPI for Document Processing Pipelines

Why File Uploads Matter in AI Applications?

AI systems require external data as input: the source material lives in the user's documents

Common Application

- Document QA Systems
- Resume Screening
- Contract Analysis
- OCR Applications
- Data Analytics

Supported File Types

-  PDF
-  DOCX
-  Images
-  CSV / Excel

File Uploads in FastAPI

```
upload.py

from fastapi import FastAPI, UploadFile

app = FastAPI()

@app.post("/upload")
async def upload(file: UploadFile):
    data = await file.read()
    return {
        "filename": file.filename,
        "size": len(data),
    }
```

Memory-efficient

Streams from a spooled temp file.

Async support

Read with `await file.read()`.

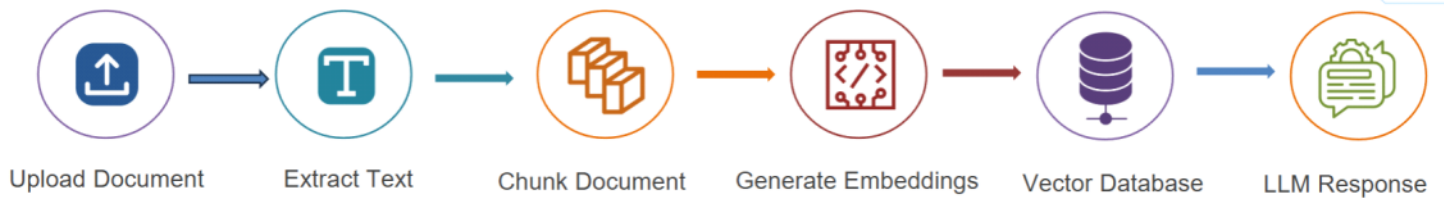
Large-file handling

Big uploads never load fully into RAM.

Metadata access

Filename, content type and size.

Document Processing Pipeline



Best Practices

File Validation

Check Type And Content Before Processing.

Size Limits

Reject Oversized Uploads Early.

Background Processing

Run Heavy Work Off The Request Thread.

Secure Storage

Store Files Safely With Access Control.

Running And Testing FastAPI With uvicorn and Swagger UI

Running FastAPI with uvicorn

uvicorn

The **ASGI server** that runs your `async` FastAPI app and serves requests.

```
Terminal
$ uvicorn main:app --reload
```

Flag	Purpose
<code>--reload</code>	Auto-restart the server on code changes: development only.
<code>--host</code>	Set the bind address, e.g. <code>0.0.0.0</code> to accept external connections.
<code>--port</code>	Set the port to listen on, e.g. 8000.

Interactive API Documentation



Swagger UI

Interactive: send live requests
at `/docs`



ReDoc

Clean read-only
reference at `/redoc`

Where It Comes From:

Both pages are **auto-generated from the OpenAPI spec** that FastAPI builds out of your routes and Pydantic models: no manual documentation to maintain.

Demonstration 1 : Building Your First FastAPI LLM Endpoint

Testing FastAPI APIs

What to test?

- ! Request validation
- ! Response models
- ! Status codes
- ! Error handling

Testing Workflow

Run Server

Open Swagger UI

Send Request

Verify Response

```
uvicorn main:app --reload
```

URL

```
http://127.0.0.1:8000/docs
```

Request Example

```
POST /chat
{
  "prompt": "Explain RAG"
}
```

Response Example

```
<> JSON
{
  "answer": "RAG combines retrieval and generation."
}
```

Summary

In this lesson, you have learned to:

- e! Understand REST APIs and FastAPI application architecture.
- e! Create validated APIs using `pydantic` request and response models.
- e! Build asynchronous and streaming endpoints for AI applications.
- e! Handle file uploads for AI document processing pipelines.
- e! Run, test, and document FastAPI applications with `uvicorn` and Swagger UI.